

07 - WiFi Provisioning + Robust Connection Handling - Session Notes

WiFi Provisioning + Robust Connection Handling

1. How WiFi Works on the ESP32

The ESP32 has a built-in WiFi radio — no external chip needed. The ESP-IDF WiFi stack manages everything: antenna, protocol, authentication, DHCP, IP address.

Your firmware does not talk to the radio directly. You talk to the WiFi driver using an event-driven API.

PC Analogy

YOUR PC WiFi	ESP32 WiFi
+-----+	+-----+
Windows Network Manager	ESP-IDF WiFi driver
Controls the WiFi card	Controls the radio
You click "Connect"	You call esp_wifi_connect
Windows handles the rest	Driver handles the rest
Windows shows you:	ESP-IDF sends you events:
"Connected"	WIFI_EVENT_STA_CONNECTED
"IP address acquired"	IP_EVENT_STA_GOT_IP
"Disconnected"	WIFI_EVENT_STA_DISCONNECTED
+-----+	+-----+

The WiFi Connection Sequence

```
Your code calls esp_wifi_connect()
      |
      v
ESP32 scans for the SSID
      |
      v
ESP32 authenticates (WPA2 handshake)
      |
      v
WIFI_EVENT_STA_CONNECTED fires
(connection to router — but NO IP yet)
      |
      v
DHCP client requests IP from router
      |
      v
IP_EVENT_STA_GOT_IP fires
(NOW you have an IP — network is ready)
```

|
v

Your code can open sockets, connect to MQTT, make HTTP requests

Critical point: `WIFI_EVENT_STA_CONNECTED` does NOT mean you can use the network. You must wait for `IP_EVENT_STA_GOT_IP`. This is the most common WiFi bug in beginner ESP32 code.

2. The WiFi Event System

ESP-IDF WiFi is entirely event-driven. You register handlers and the driver calls them when things happen.

PC Analogy

Windows Network Events	ESP-IDF WiFi Events
+-----+	+-----+
Network connected	WIFI_EVENT_STA_CONNECTED
IP address assigned	IP_EVENT_STA_GOT_IP
Network disconnected	WIFI_EVENT_STA_DISCONNECTED
Connection failed	WIFI_EVENT_STA_DISCONNECTED
	(reason = AUTH_FAIL)
+-----+	+-----+

Windows tells your programs via notifications.
ESP-IDF tells your firmware via event callbacks.
Same concept. Different API.

All WiFi Events You Need to Handle

Event	When it fires	What to do
WIFI_EVENT_STA_START	WiFi driver started	Call <code>esp_wifi_connect()</code>
WIFI_EVENT_STA_CONNECTED	Associated with AP	Wait for IP
IP_EVENT_STA_GOT_IP	DHCP assigned IP	NOW start your app
WIFI_EVENT_STA_DISCONNECTED	Lost connection	Try to reconnect
WIFI_EVENT_STA_AUTHMODE_CHANGE	Security changed	Handle gracefully

Basic Event Handler

```
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_netif.h"
#include "esp_log.h"

static const char *TAG = "wifi";

static void wifi_event_handler(void *arg,
```

```

        esp_event_base_t event_base,
        int32_t event_id,
        void *event_data)
{
    if (event_base == WIFI_EVENT) {

        switch (event_id) {

            case WIFI_EVENT_STA_START:
                ESP_LOGI(TAG, "WiFi started - connecting...");
                esp_wifi_connect();
                break;

            case WIFI_EVENT_STA_CONNECTED:
                ESP_LOGI(TAG, "Connected to AP - waiting for IP...");
                break;

            case WIFI_EVENT_STA_DISCONNECTED:
                ESP_LOGW(TAG, "Disconnected - will retry...");
                esp_wifi_connect(); /* Simple reconnect */
                break;

        }

    } else if (event_base == IP_EVENT) {

        if (event_id == IP_EVENT_STA_GOT_IP) {
            ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
            ESP_LOGI(TAG, "Got IP: " IPSTR,
                     IP2STR(&event->ip_info.ip));
            /* NOW safe to start network operations */
        }

    }

}

```

3. First WiFi Connection — Full Code Pattern

The Initialisation Sequence

Every ESP32 WiFi project must do these steps in order:

```

static void wifi_init(const char *ssid, const char *password)
{
    /* Step 1 - Initialise the network interface layer */
    ESP_ERROR_CHECK(esp_netif_init());

    /* Step 2 - Create the default event loop */
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    /* Step 3 - Create default WiFi station netif */
    esp_netif_create_default_wifi_sta();

    /* Step 4 - Initialise WiFi driver with default config */

```

```

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&cfg));

/* Step 5 – Register event handlers */
ESP_ERROR_CHECK(esp_event_handler_register(
    WIFI_EVENT, ESP_EVENT_ANY_ID, &wifi_event_handler, NULL));
ESP_ERROR_CHECK(esp_event_handler_register(
    IP_EVENT, IP_EVENT_STA_GOT_IP, &wifi_event_handler, NULL));

/* Step 6 – Configure SSID and password */
wifi_config_t wifi_config = {
    .sta = {
        .threshold.authmode = WIFI_AUTH_WPA2_PSK,
    },
};
strncpy((char *)wifi_config.sta.ssid, ssid,
    sizeof(wifi_config.sta.ssid) - 1);
strncpy((char *)wifi_config.sta.password, password,
    sizeof(wifi_config.sta.password) - 1);

/* Step 7 – Set station mode and apply config */
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));

/* Step 8 – Start WiFi driver
    WIFI_EVENT_STA_START fires -> handler calls esp_wifi_connect() */
ESP_ERROR_CHECK(esp_wifi_start());

ESP_LOGI(TAG, "WiFi init complete – connecting to %s", ssid);
}

```

4. Why You Should Never Hardcode Credentials

The Problem

```

/* NEVER do this in production firmware */
#define WIFI_SSID      "MyHomeNetwork"
#define WIFI_PASSWORD "MyPassword123"

```

Why this is wrong:

Problem	Consequence
Password in source code	Visible in GitHub, code reviews, leaks
Password in binary	Can be extracted from the .bin file with esptool
Different SSID per customer	Need a different firmware build per customer
Customer changes router	Device can never reconnect — needs reflashing
Office WiFi changes	Every deployed device breaks simultaneously

The Solution — NVS Storage

NVS (Non-Volatile Storage) is the ESP32's equivalent of a configuration file that survives power off and reboot.

PC Analogy:

Hardcoded credentials = username/password typed in source code

NVS credentials = credentials stored in Windows Credential Manager
Encrypted, persistent, user can change them

Storing Credentials in NVS

```
#include "nvs_flash.h"
#include "nvs.h"

#define NVS_NAMESPACE    "wifi_config"
#define NVS_KEY_SSID     "ssid"
#define NVS_KEY_PASSWORD "password"

static esp_err_t wifi_credentials_save(const char *ssid,
                                       const char *password)
{
    nvs_handle_t handle;

    /* Open NVS namespace for writing */
    esp_err_t ret = nvs_open(NVS_NAMESPACE, NVS_READWRITE, &handle);
    if (ret != ESP_OK) return ret;

    /* Write SSID */
    ret = nvs_set_str(handle, NVS_KEY_SSID, ssid);
    if (ret != ESP_OK) goto cleanup;

    /* Write password */
    ret = nvs_set_str(handle, NVS_KEY_PASSWORD, password);
    if (ret != ESP_OK) goto cleanup;

    /* Commit - writes to flash */
    ret = nvs_commit(handle);

cleanup:
    nvs_close(handle);
    return ret;
}

static esp_err_t wifi_credentials_load(char *ssid, size_t ssid_len,
                                       char *password, size_t pass_len)
{
    nvs_handle_t handle;

    esp_err_t ret = nvs_open(NVS_NAMESPACE, NVS_READONLY, &handle);
    if (ret != ESP_OK) return ret;

    ret = nvs_get_str(handle, NVS_KEY_SSID, ssid, &ssid_len);
```

```

    if (ret != ESP_OK) goto cleanup;

    ret = nvs_get_str(handle, NVS_KEY_PASSWORD, password, &pass_len);

cleanup:
    nvs_close(handle);
    return ret;
}

```

5. Robust Reconnection — The Right Way

What Happens When WiFi Drops

```

Device is running happily
    |
    v
Router reboots or signal drops
    |
    v
WIFI_EVENT_STA_DISCONNECTED fires
    |
    v
What do you do?

BAD: Call esp_wifi_connect() immediately in a loop
     Hammers the router with connection attempts
     Wastes power
     Router may block the device (too many attempts)

GOOD: Exponential backoff
      Wait 1s, try again
      Wait 2s, try again
      Wait 4s, try again
      Wait 8s, try again
      Wait 16s, try again
      Cap at 60s maximum
      Keep trying — never give up

```

Exponential Backoff Pattern

```

static int  s_retry_count      = 0;
static int  s_retry_delay_ms   = 1000; /* Start at 1 second */
#define     MAX_RETRY_DELAY    60000 /* Cap at 60 seconds */
#define     BACKOFF_MULTIPLIER 2

static void handle_disconnected(void)
{
    s_retry_count++;

    ESP_LOGW(TAG, "Disconnected. Retry #%d in %dms",
              s_retry_count, s_retry_delay_ms);
}

```

```

/* Wait before retrying */
vTaskDelay(pdMS_TO_TICKS(s_retry_delay_ms));

/* Double the delay for next attempt – exponential backoff */
s_retry_delay_ms *= BACKOFF_MULTIPLIER;
if (s_retry_delay_ms > MAX_RETRY_DELAY) {
    s_retry_delay_ms = MAX_RETRY_DELAY;
}

/* Attempt reconnection */
esp_wifi_connect();
}

static void handle_connected(void)
{
    /* Reset retry state on successful connection */
    s_retry_count = 0;
    s_retry_delay_ms = 1000;
    ESP_LOGI(TAG, "Reconnected successfully after %d attempts",
              s_retry_count);
}

```

PC Analogy for Exponential Backoff

Think of calling customer support:

First try – busy – wait 1 minute – try again

Second try – busy – wait 2 minutes – try again

Third try – busy – wait 4 minutes – try again

If you called every second – you would clog the phone line.

If you wait longer each time – you are polite and efficient.

Routers work the same way.

6. EventGroup — Notifying Tasks That WiFi Is Ready

Multiple tasks in your firmware may need to wait for WiFi. The cleanest way is an EventGroup bit.

```

#include "freertos/event_groups.h"

static EventGroupHandle_t s_wifi_event_group;

#define WIFI_CONNECTED_BIT BIT0
#define WIFI_FAIL_BIT BIT1

/* In event handler – set bit when IP acquired */
case IP_EVENT_STA_GOT_IP:
    xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
    break;

/* In your application task – wait for WiFi before proceeding */

```

```

void mqtt_task(void *arg)
{
    /* Block here until WiFi is ready */
    EventBits_t bits = xEventGroupWaitBits(
        s_wifi_event_group,
        WIFI_CONNECTED_BIT,
        pdFALSE,          /* Don't clear bit after reading */
        pdTRUE,           /* Wait for ALL bits (just one here) */
        portMAX_DELAY     /* Wait forever */
    );

    if (bits & WIFI_CONNECTED_BIT) {
        ESP_LOGI(TAG, "WiFi ready - starting MQTT");
        /* Start MQTT connection */
    }
}

```

PC Analogy for EventGroup

EventGroup = A bulletin board in the office

WiFi manager posts a notice: "INTERNET IS UP"

All tasks waiting for internet see the notice and start working

Tasks do not poll the WiFi status constantly.

They wait at the bulletin board.

The moment the notice goes up – they all start.

7. WiFi Provisioning — Letting Users Configure Credentials

For a real product — the user must be able to set their own WiFi credentials without reflashing firmware.

Three Provisioning Methods

METHOD 1 — SmartConfig (deprecated but simple)

User installs app on phone

App sends SSID + password over UDP broadcast

ESP32 sniffs the packets and extracts credentials

Works: sometimes

Reliability: poor

Status: deprecated by Espressif

METHOD 2 — BLE Provisioning (recommended)

ESP32 creates a BLE advertisement

User connects via Espressif BLE provisioning app

App sends credentials over encrypted BLE

ESP32 saves to NVS and connects

Works: very reliably

Reliability: excellent

Status: recommended by Espressif

METHOD 3 – SoftAP Provisioning (most universal)
ESP32 creates its own WiFi hotspot
User connects phone to ESP32 hotspot
User opens a web browser and goes to 192.168.4.1
User types SSID and password in a web form
ESP32 saves to NVS and connects to real network
Works: on any device with a browser
Reliability: excellent
Status: widely used in production

SoftAP Provisioning Flow

```
Factory fresh device powers on
|
v
No credentials in NVS
|
v
ESP32 starts SoftAP mode
Creates hotspot: "ESP32-Setup-XXXX"
|
v
User connects phone to "ESP32-Setup-XXXX"
|
v
User opens browser -> 192.168.4.1
|
v
User enters SSID and password in web form
|
v
ESP32 receives credentials
Saves to NVS
Restarts in Station mode
|
v
ESP32 connects to user's router
Operates normally forever
|
v
If user needs to reconfigure:
Hold button for 5 seconds -> clear NVS -> restart SoftAP
```

Simple SoftAP Provisioning Credential Check

```
void app_main(void)
{
    /* Initialise NVS first - always */
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();
        nvs_flash_init();
    }
}
```

```

}

/* Try to load saved credentials */
char ssid[32] = {0};
char password[64] = {0};

esp_err_t cred_ret = wifi_credentials_load(
    ssid, sizeof(ssid),
    password, sizeof(password)
);

if (cred_ret == ESP_OK && strlen(ssid) > 0) {
    /* Credentials found – connect directly */
    ESP_LOGI(TAG, "Credentials found – connecting to %s", ssid);
    wifi_init_sta(ssid, password);
} else {
    /* No credentials – start provisioning */
    ESP_LOGI(TAG, "No credentials – starting provisioning");
    wifi_start_provisioning();
}
}

```

8. Production WiFi Manager — Complete Pattern

```

+-----+
|          PRODUCTION WiFi MANAGER          |
|                                             |
| app_main()                                |
| |                                           |
| +--> nvs_flash_init()                      |
| +--> load credentials from NVS             |
| |                                           |
| +--> credentials found?                   |
|     YES -> wifi_init_sta()                 |
|     NO  -> wifi_start_provisioning()       |
|                                             |
| wifi_event_handler()                      |
|     WIFI_EVENT_STA_START                  |
|         -> esp_wifi_connect()              |
|     WIFI_EVENT_STA_CONNECTED              |
|         -> log, wait for IP                |
|     IP_EVENT_STA_GOT_IP                  |
|         -> set EventGroup CONNECTED bit    |
|         -> reset retry counter             |
|     WIFI_EVENT_STA_DISCONNECTED           |
|         -> clear EventGroup CONNECTED bit  |
|         -> exponential backoff retry       |
|                                             |
| Application tasks                          |
|     Wait on EventGroup before starting    |

```

```
| Re-check EventGroup if network needed |  
+-----+  
|
```

9. Checking WiFi Signal Strength — RSSI

In production you want to know how strong the WiFi signal is. Useful for placement advice to customers.

```
static void log_wifi_rssi(void)  
{  
    wifi_ap_record_t ap_info;  
    esp_err_t ret = esp_wifi_sta_get_ap_info(&ap_info);  
  
    if (ret == ESP_OK) {  
        ESP_LOGI(TAG, "RSSI: %d dBm", ap_info.rssi);  
  
        /* Interpret signal strength */  
        if (ap_info.rssi >= -50) {  
            ESP_LOGI(TAG, "Signal: Excellent");  
        } else if (ap_info.rssi >= -60) {  
            ESP_LOGI(TAG, "Signal: Good");  
        } else if (ap_info.rssi >= -70) {  
            ESP_LOGI(TAG, "Signal: Fair");  
        } else {  
            ESP_LOGW(TAG, "Signal: Weak - consider moving closer");  
        }  
    }  
}
```

RSSI Reference Table

RSSI Value	Signal Quality	Use Case
-30 to -50 dBm	Excellent	Device right next to router
-50 to -60 dBm	Good	Normal room distance
-60 to -70 dBm	Fair	Far room or weak router
-70 to -80 dBm	Weak	Borderline — may drop
Below -80 dBm	Very weak	Will disconnect frequently

Summary Table

Concept	What it is	PC Equivalent
WiFi Station mode	ESP32 connects to a router	Your PC connecting to WiFi
WiFi SoftAP mode	ESP32 becomes the router	Your PC creating a hotspot

Concept	What it is	PC Equivalent
esp_netif_init	Network interface layer init	Winsock initialisation
WIFI_EVENT_STA_CONNECTED	Associated with AP	Windows shows WiFi bars
IP_EVENT_STA_GOT_IP	DHCP assigned IP	Windows shows IP in settings
NVS credentials	Saved WiFi config	Windows Credential Manager
Exponential backoff	Increasing retry delay	Polite retry strategy
EventGroup	Notify tasks WiFi is ready	Bulletin board notice
RSSI	Signal strength in dBm	WiFi bars on your phone
Provisioning	User sets credentials	Windows WiFi setup wizard